Deep Hedging: A Walkthrough

From Neural Networks to Hedging Under Frictions

Qianmo Liu

November 4, 2026

Abstract

These notes build deep hedging (Buehler, Gonon, Teichmann and Wood, 2019) from scratch. We first explain why hedging in a realistic market is an optimization problem rather than a replication problem, then introduce the risk measures used as objectives, then the neural-network machinery, and finally put everything together into a training algorithm. The last sections cover the market simulators, benchmarks, expected results, and an implementation guide matched to the project repository. Each section states the technique first and then works through a concrete example.

Contents

1 Setup and Notation **2**

2 Why Hedging Is an Optimization Problem **2**
2.1 Technique . . . 2
2.2 Example: a one-period trinomial market . . . 3

3 Risk Measures **3**
3.1 Technique . . . 3
3.2 CVaR as an optimization: the Rockafellar–Uryasev trick . . . 4
3.3 Entropic risk . . . 5
3.4 Indifference pricing . . . 5

4 Neural Networks **6**
4.1 From linear regression to an MLP . . . 6
4.2 Training: stochastic gradients . . . 7
4.3 Backpropagation . . . 9
4.4 Practical rules . . . 9

5 Deep Hedging **9**
5.1 From strategies to networks . . . 9
5.2 The rollout . . . 10
5.3 How the gradient flows . . . 11
5.4 The algorithm . . . 11

6 Market Simulators **12**
6.1 Geometric Brownian motion . . . 12
6.2 Heston . . . 12
6.3 The variance swap as a second instrument . . . 12

7 Benchmarks and What to Expect **13**
7.1 GBM without costs: recover the BS delta . . . 13
7.2 GBM with costs: the no-trade band . . . 13
7.3 Heston . . . 14
7.4 Indifference price . . . 14

8 Experiments and Evaluation **14**

9 Implementation Guide **15**
9.1 Repository map . . . 15
9.2 Interfaces . . . 15
9.3 Tests . . . 15
9.4 Debugging checklist . . . 16

10 Limitations and Extensions **16**

1 Setup and Notation

Throughout we work on a filtered probability space $(\Omega, \mathcal{F}, (\mathcal{F}_k)_{k=0}^n, \mathbb{P})$ with trading dates

$$0 = t_0 < t_1 < \cdots < t_n = T, \qquad t_k = k\,\Delta t.$$

- $S_k = (S_k^{(1)}, \ldots, S_k^{(m)}) \in \mathbb{R}^m$: prices of the m hedging instruments at t_k.
- Z: the liability we are short, an $\mathcal{F}_n$-measurable random variable. Running example: a European call, $Z = (S_n^{(1)} - K)^+$. Note that this is very important for the hedging concept since as a seller, the loss is unbounded.
- $\delta_k \in \mathbb{R}^m$: number of units of each instrument held over $[t_k, t_{k+1})$, chosen at t_k. A strategy $\delta = (\delta_0, \ldots, \delta_{n-1})$ is *admissible* if each δ_k is $\mathcal{F}_k$-measurable. We set $\delta_{-1} = \delta_n = 0$ (start flat, liquidate at maturity).
- Interest rate $r = 0$. Every price is already in units of the bank account, so cash earns nothing.
- Proportional transaction cost $c \geq 0$: trading Δ units at price S costs $cS|\Delta|$.

Definition 1.1 (Hedging P&L)**.** For an initial premium p_0 received for selling Z, the terminal profit and loss is

$$\mathrm{PL}_T(\delta) = -Z + p_0 + \underbrace{\sum_{k=0}^{n-1} \delta_k \cdot (S_{k+1} - S_k)}_{(\delta \cdot S)_T \text{ (hedging gains)}} - \underbrace{\sum_{k=0}^{n} \sum_{j=1}^{m} c\, S_k^{(j)} \left|\delta_k^{(j)} - \delta_{k-1}^{(j)}\right|}_{C_T(\delta) \text{ (trading costs)}}. \tag{1}$$

Remark 1.2. Equation (1) yields a self-financing process: every cost is paid out of cash, and every gain or loss on the instruments accures to cash.

2 Why Hedging Is an Optimization Problem

2.1 Technique

In the Black–Scholes model with continuous trading and no costs, the market is *complete*: there exist p_0 and δ with $\mathrm{PL}_T(\delta) = 0$ almost surely, namely the BS price and $\delta_t = N(d_1)$. Replication fails as soon as any of the following holds:

1. **Discrete trading.** The hedge is constant on $[t_k, t_{k+1})$, so that using δ_k shares of stocks to approximate the convex option value leaves a second-order error on each interval.

   $$\varepsilon_k \approx \tfrac{1}{2}\,\Gamma_k \big[(\Delta S_k)^2 - \sigma^2 S_k^2\,\Delta t\big],$$

   which is of order Δt with mean approximately zero. Summing $n = T/\Delta t$ such roughly independent errors gives a total hedging error with standard deviation of order $\sqrt{n}\,\Delta t = \sqrt{T\Delta t}$. The error vanishes only in the limit $\Delta t \to 0$.

2. **Transaction costs.** Continuous delta hedging has infinite total variation, so its cost is infinite. Hence this disagrees with the conclusion above.

3. **Untraded risk factors.** Under Heston with only the stock traded, volatility risk cannot be completely hedged.

When replication is impossible, there is no "correct" hedge. We must instead pick a criterion that ranks the random outcomes $\mathrm{PL}_T(\delta)$, and optimize it:

$$\pi(-Z) \;=\; \inf_{\delta \text{ admissible}} \rho\big(-Z + (\delta \cdot S)_T - C_T(\delta)\big), \tag{2}$$

where ρ is a risk measure (Section 3), and $\pi(-Z)$ denote how much cash must be added to X under the optimal hedging to make the position acceptable.

2.2 Example: a one-period trinomial market

Let $S_0 = 100$, $S_1 \in \{90, 100, 110\}$ with probability $1/3$ each, $K = 100$, no costs. Then $Z \in \{0, 0, 10\}$.

No replication. We would need $p_0 + \delta(S_1 - 100) = Z$ in every state:

$$p_0 - 10\delta = 0, \qquad p_0 = 0, \qquad p_0 + 10\delta = 10.$$

The first two equations give $p_0 = \delta = 0$, which contradicts the third. Three states, two unknowns.

Quadratic-optimal hedge. Minimize $\mathbb{E}[(p_0 + \delta\Delta S - Z)^2]$ with $\Delta S = S_1 - S_0$. This is a linear regression of Z on ΔS:

$$\delta^* = \frac{\mathrm{Cov}(Z, \Delta S)}{\mathrm{Var}(\Delta S)} = \frac{\frac{1}{3}(10 \cdot 10)}{\frac{1}{3}(100 + 0 + 100)} = \frac{1}{2}, \qquad p_0^* = \mathbb{E}[Z] = \frac{10}{3}.$$

The residual PL is $\{-\frac{5}{3},\ \frac{10}{3},\ -\frac{5}{3}\}$, with variance $\frac{50}{9}$. This variance is the irreducible, unhedgeable risk. A different ρ (for instance CVaR) would penalize the three outcomes differently and give a different δ^*.

Deep hedging does exactly this, but with many periods, many instruments, and frictions, where no closed-form regression exists.

3 Risk Measures

3.1 Technique

Definition 3.1 (Convex risk measure)**.** A map ρ from random variables (P&Ls) to $\mathbb{R}$ is a *convex risk measure* if for all X, Y:

(i) Monotonicity: $X \le Y \Rightarrow \rho(X) \ge \rho(Y)$. More profit means less risk.

(ii) Convexity: $\rho(\lambda X + (1-\lambda)Y) \le \lambda\rho(X) + (1-\lambda)\rho(Y)$ for $\lambda \in [0, 1]$. Diversification does not increase risk.

(iii) Cash invariance: $\rho(X + c) = \rho(X) - c$ for constants c. Adding cash reduces risk one-for-one.

Cash invariance means $\rho(X)$ is the smallest amount of cash that must be added to X to make the position acceptable. There are three objectives used in the project:

Name	Formula	Comment
Quadratic	$\mathbb{E}[X^2]$	Not a risk measure. Penalizes gains and losses equally.
Entropic	$\rho_\lambda(X) = \frac{1}{\lambda}\log \mathbb{E}\big[e^{-\lambda X}\big]$	Exponential utility. $\lambda > 0$ is risk aversion.
CVaR_α	$\min_{w\in\mathbb{R}}\Big\{w + \frac{1}{1-\alpha}\mathbb{E}\big[(-X-w)^+\big]\Big\}$	Mean of the worst $(1-\alpha)$ fraction of losses.

3.2 CVaR as an optimization: the Rockafellar–Uryasev trick

The naive definition of CVaR requires sorting the sample to find the α-quantile, which is awkward to differentiate. The following representation turns it into a smooth-enough minimization.

Proposition 3.2 (Rockafellar–Uryasev)**.** *Let $L = -X$ be the loss, with continuous distribution. Define*

$$G(w) = w + \frac{1}{1-\alpha}\mathbb{E}\big[(L-w)^+\big].$$

Then G is convex, it is minimized at $w^* = \mathrm{VaR}_\alpha(L)$ (the α-quantile of L), and

$$\min_w G(w) = \mathbb{E}\big[L \,\big|\, L \geq \mathrm{VaR}_\alpha(L)\big] = \mathrm{CVaR}_\alpha(X).$$

Proof. G is convex because $w \mapsto (L-w)^+$ is convex and expectation preserves convexity. Differentiating,

$$G'(w) = 1 - \frac{1}{1-\alpha}\,\mathbb{P}(L > w),$$

which is zero exactly when $\mathbb{P}(L > w) = 1-\alpha$, i.e. $w^* = \mathrm{VaR}_\alpha(L)$. Substituting w^*:

$$G(w^*) = w^* + \frac{\mathbb{E}[(L-w^*)\mathbf{1}\{L > w^*\}]}{\mathbb{P}(L > w^*)} = w^* + \mathbb{E}[L - w^* \mid L > w^*] = \mathbb{E}[L \mid L > w^*]. \qquad \square$$

Remark 3.3 (Why this matters for deep hedging). Since $\mathrm{CVaR}_\alpha(X) = \min_w G(w)$, minimizing CVaR over the strategy is the same as

$$\min_\theta \ \min_w \ G_\theta(w) = \min_{(\theta,w)} G_\theta(w).$$

So w becomes *one extra trainable scalar*, optimized jointly with the network weights θ by the same gradient method. At the optimum, w is the VaR of the hedged position, which is useful as a free diagnostic.

Example 3.4 (CVaR by hand)**.** Five equally likely losses $L \in \{-1, 0, 2, 3, 6\}$, $\alpha = 0.6$. The worst 40% are $\{3, 6\}$, so CVaR $= 4.5$. Using G, with $\frac{1}{1-\alpha} = 2.5$:

$$\begin{aligned} G(2) &= 2 + 2.5 \cdot \tfrac{1}{5}(0+0+0+1+4) = 4.5, \\ G(3) &= 3 + 2.5 \cdot \tfrac{1}{5}(0+0+0+0+3) = 4.5, \\ G(4) &= 4 + 2.5 \cdot \tfrac{1}{5}(0+0+0+0+2) = 5.0. \end{aligned}$$

The minimum is 4.5, as expected. With a discrete sample, G is flat between neighboring order statistics. The sample has only 5 points, so this is an extreme case of the same effect that makes small batches noisy in training: the CVaR gradient only comes from the $(1-\alpha)B$ worst paths.

3.3 Entropic risk

$\rho_\lambda(X) = \frac{1}{\lambda}\log \mathbb{E}[e^{-\lambda X}]$ is cash invariant: $\rho_\lambda(X+c) = \frac{1}{\lambda}\log\big(e^{-\lambda c}\mathbb{E}[e^{-\lambda X}]\big) = \rho_\lambda(X) - c$. For $X \sim N(\mu, s^2)$ it equals $-\mu + \frac{\lambda}{2}s^2$, which makes the trade-off between mean and variance explicit. In practice, $e^{-\lambda X}$ overflows easily, so compute it with `torch.logsumexp(-lam*X, 0) - log(B)`, divided by λ.

3.4 Indifference pricing

Definition 3.5 (Indifference price)**.** Let π be defined as in (2). The seller's *indifference price* of Z is

$$p_0(Z) \;=\; \pi(-Z) - \pi(0).$$

Remark 3.6 (Interpretation). Compare the seller's optimal risk in two situations:

- **Sell Z for a premium p and hedge optimally.** By cash invariance, the optimal risk is $\pi(-Z) - p$, which decreases one-for-one in p.
- **Do not sell, and only trade the hedging instruments.** The optimal risk is $\pi(0)$, which does not depend on p.

$p_0(Z)$ is the unique premium that makes the two risks equal:

$$\pi(-Z) - p_0 = \pi(0) \quad\Longleftrightarrow\quad p_0 = \pi(-Z) - \pi(0).$$

The seller is therefore indifferent between selling and not selling at $p = p_0(Z)$. Selling is preferable if $p > p_0(Z)$, and not selling is preferable if $p < p_0(Z)$. Thus $p_0(Z)$ is the minimum acceptable premium. It assumes the seller hedges with the optimal strategy, and it depends on the chosen ρ and on the market model.

Remark 3.7 (The benchmark $\pi(0)$). Since $\delta \equiv 0$ is admissible and $\rho(0) = 0$, we always have $\pi(0) \le 0$. Trading may reduce risk further when the instruments have nonzero expected returns under $\mathbb{P}$.

If S is a $\mathbb{P}$-martingale and there are no transaction costs, then $\mathbb{E}[(\delta\cdot S)_T] = 0$ for every admissible δ. For the risk measures used here, $\rho(X) \ge -\mathbb{E}[X]$, so $\pi(0) \ge 0$, and hence $\pi(0) = 0$. With proportional costs, trading only adds a nonpositive term, so $\pi(0) = 0$ still holds. In this setting $p_0(Z) = \pi(-Z)$.

Proposition 3.8 (The premium drops out)**.** *If ρ is cash invariant, then for any constant p_0*

$$\arg\min_\delta \rho\big(p_0 - Z + (\delta\cdot S)_T - C_T(\delta)\big) = \arg\min_\delta \rho\big(-Z + (\delta\cdot S)_T - C_T(\delta)\big).$$

Proof. By cash invariance, the left objective equals the right objective minus p_0, a constant that does not depend on δ. □

So under entropic risk or CVaR we train *without* a premium, and get the price afterwards from two training runs: one with Z and one with $Z = 0$.

Remark 3.9 (Quadratic loss). $\mathbb{E}[X^2]$ is not cash invariant, so the premium matters. Make p_0 a trainable scalar: minimizing $\mathbb{E}[(p_0 + Y)^2]$ over p_0 gives $p_0 = -\mathbb{E}[Y]$, so the remaining objective is $\mathrm{Var}(Y)$. In the trinomial example above, this is exactly why $p_0^* = \mathbb{E}[Z]$.

4 Neural Networks

4.1 From linear regression to an MLP

Linear model. $\hat{y} = w^\top x + b$, fitted by minimizing $L(w,b) = \frac{1}{N}\sum_i (\hat{y}_i - y_i)^2$. Instead of the normal equations, we can use *gradient descent*:

$$\theta \leftarrow \theta - \eta\, \nabla_\theta L(\theta), \qquad \theta = (w,b),\ \eta > 0 \text{ the learning rate.}$$

A neural network replaces the linear function with a more flexible one. The fitting method is unchanged.

Proposition 4.1 (Steepest descent direction)**.** *Let $L : \mathbb{R}^p \to \mathbb{R}$ be differentiable at θ with $\nabla L(\theta) \neq 0$. Among all unit directions u ($\|u\| = 1$), the directional derivative*

$$D_u L(\theta) = \lim_{h \downarrow 0} \frac{L(\theta + hu) - L(\theta)}{h} = \nabla L(\theta) \cdot u$$

is maximized by $u^* = \nabla L(\theta)/\|\nabla L(\theta)\|$ and minimized by $-u^*$, with extreme values $\pm\|\nabla L(\theta)\|$.

Proof. By the Cauchy–Schwarz inequality,

$$-\|\nabla L(\theta)\| = -\|\nabla L(\theta)\|\,\|u\| \ \le\ \nabla L(\theta)\cdot u \ \le\ \|\nabla L(\theta)\|\,\|u\| = \|\nabla L(\theta)\|,$$

with equality on the right if and only if u is a positive multiple of $\nabla L(\theta)$, and on the left if and only if u is a negative multiple. □

Remark 4.2 (Gradient descent)*.* The proposition motivates the update

$$\theta \leftarrow \theta - \eta\, \nabla L(\theta), \qquad \eta > 0,$$

which moves in the locally steepest descent direction. By a first-order Taylor expansion,

$$L\big(\theta - \eta \nabla L(\theta)\big) = L(\theta) - \eta\,\|\nabla L(\theta)\|^2 + o(\eta),$$

so the loss decreases for sufficiently small η. The guarantee is only local: if η is too large, the $o(\eta)$ term dominates and the loss may increase. To maximize an objective, apply the same update to its negative.

Why nonlinearity is needed. Stacking linear maps gives a linear map:

$$W_2(W_1 x + b_1) + b_2 = (W_2 W_1)x + (W_2 b_1 + b_2).$$

So we insert an elementwise nonlinearity σ after each layer, and this σ function is referred to as *the activation function*. The standard choice is $\mathrm{ReLU}(z) = \max(z, 0)$. A smooth alternative is $\mathrm{SiLU}(z) = z/(1 + e^{-z})$.

Definition 4.3 (Multilayer perceptron)**.** An MLP with L hidden layers of widths $h_1, \ldots, h_L$ is

$$a_0 = x, \qquad a_\ell = \sigma(W_\ell a_{\ell-1} + b_\ell)\ \ (\ell = 1, \ldots, L), \qquad F_\theta(x) = W_{L+1} a_L + b_{L+1},$$

with $W_\ell \in \mathbb{R}^{h_\ell \times h_{\ell-1}}$, $b_\ell \in \mathbb{R}^{h_\ell}$, $h_0 = d$ (input dimension), $h_{L+1} = m$ (output dimension). The parameter vector is $\theta = (W_1, b_1, \ldots, W_{L+1}, b_{L+1})$. The output layer has no activation.

Example 4.4 (Counting parameters)**.** Input $d = 3$, two hidden layers of width 64, output $m = 1$:

$$\underbrace{64 \cdot 3 + 64}_{256} + \underbrace{64 \cdot 64 + 64}_{4160} + \underbrace{1 \cdot 64 + 1}_{65} = 4481.$$

Each neuron has one weight per input plus one bias. Most parameters sit in the 64×64 middle layer.

Theorem 4.5 (Universal approximation, informal; Hornik 1991)**.** *Let σ be continuous and non-polynomial. For every continuous f on a compact set $\mathcal{K} \subset \mathbb{R}^d$ and every $\varepsilon > 0$, there is a one-hidden-layer network F_θ with $\sup_{x \in \mathcal{K}} |F_\theta(x) - f(x)| < \varepsilon$.*

This is the reason a network can stand in for an unknown hedging rule: whatever the optimal $\delta_k = f_k^*(I_k)$ is, a large enough network can approximate it. The theorem says nothing about how to *find* θ; that is the job of training.

4.2 Training: stochastic gradients

The problem. In most cases the loss is an expectation, $L(\theta) = \mathbb{E}[\ell(\theta; \xi)]$, with ξ random: a data point drawn uniformly from a finite dataset (so that the expectation is the dataset average), or in deep hedging a simulated path. This holds for the quadratic loss and for CVaR in the Rockafellar–Uryasev form; entropic risk is a nonlinear function of an expectation.

For a large dataset, computing this average at every step is expensive. In deep hedging, ξ ranges over infinitely many paths, so the exact gradient cannot be computed at all.

The idea. At each step, draw a *mini-batch* $\xi_1, \ldots, \xi_B$ of B samples (the *batch size*) and replace the expectation by a Monte Carlo average:

$$\widehat{\nabla L}(\theta) = \frac{1}{B} \sum_{i=1}^{B} \nabla_\theta \ell(\theta; \xi_i), \qquad \theta_{k+1} \leftarrow \theta_k - \eta_k \, \widehat{\nabla L}(\theta_k).$$

Since $\mathbb{E}\big[\widehat{\nabla L}(\theta)\big] = \nabla L(\theta)$, each step moves in the correct direction on average, with zero-mean noise. This is *stochastic gradient descent (SGD)*.

Terminology. One parameter update is a *step* (or *iteration*). With a fixed dataset of N points, shuffled and split into mini-batches, one pass through all the data is an *epoch*, consisting of N/B steps.

Example 4.6 (Full-batch vs. mini-batch)**.** Data $x = \{1, 2, 3, 4, 5, 6\}$, $L(\theta) = \frac{1}{6} \sum_i (\theta - x_i)^2$, minimized at $\theta^* = \bar{x} = 3.5$. The gradient of a batch is $2(\theta - \text{batch mean})$. Start at $\theta_0 = 0$ with $\eta = 0.25$.

	Full batch (all 6 points)			Mini-batch, $B = 2$		
Step	mean	gradient	θ	batch	gradient	θ
1	3.5	-7	1.75	$\{5, 1\}$	-6	1.5
2	3.5	-3.5	2.625	$\{6, 2\}$	-5	2.75
3	3.5	-1.75	3.0625	$\{3, 4\}$	-1.5	3.125

Both approach 3.5 at a similar rate, but each mini-batch step uses a third of the data. The batch means (3, 4, 3.5) differ from 3.5, so individual steps are biased in direction, but the errors average out. The three steps form one epoch.

Does the noise average out? Yes, but only partly. With a constant step size η, SGD reaches a neighbourhood of the minimizer and then keeps fluctuating inside it, because every step adds fresh noise. A smaller η gives smaller fluctuations. To converge exactly, the step size must shrink over time (for example $\eta_k \propto 1/k$). In practice we use a constant η and lower it once the loss stops improving. A classical result going back to Robbins and Monro (1951) states that if L is strongly convex, the gradient noise has bounded variance, and $\sum_k \eta_k = \infty$, $\sum_k \eta_k^2 < \infty$, then SGD converges to the minimizer almost surely.

Remark 4.7 (Neural networks). For a neural network, L is not convex. Theory then only guarantees convergence to points where $\nabla L \approx 0$, which may be local minima or saddle points. That SGD finds good solutions for wide networks is largely an empirical fact. The gradient noise can even help, by pushing the iterates out of poor local minima.

Adam. Plain SGD uses a single step size η_k for all coordinates of θ at step k. This is a problem when the gradient components have very different magnitudes: with gradient components 100 and 0.01 and $\eta = 10^{-3}$, SGD moves the first coordinate by 0.1 (possibly overshooting) and the second by 10^{-5} (essentially frozen). Adam (Kingma and Ba, 2015) fixes this by giving each coordinate its own effective step size.

For each coordinate j, Adam keeps exponential moving averages of the gradient and of its square. With $g_k = \widehat{\nabla L}(\theta_k)$ and all operations taken coordinatewise,

$$m_k = \beta_1\, m_{k-1} + (1-\beta_1)\, g_k, \qquad \hat{m}_k = \frac{m_k}{1-\beta_1^k},$$

$$v_k = \beta_2\, v_{k-1} + (1-\beta_2)\, g_k^2, \qquad \hat{v}_k = \frac{v_k}{1-\beta_2^k},$$

$$\theta_{k+1} = \theta_k - \eta\, \frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \epsilon},$$

with $m_0 = v_0 = 0$ and standard defaults $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$. The factors $1-\beta^k$ correct the bias toward 0 caused by initializing the averages at 0; they matter only in the first few hundred steps.

- $\hat{m}_k$ (*momentum*) averages recent gradients, so zero-mean noise partly cancels and a consistent direction remains.
- Dividing by $\sqrt{\hat{v}_k}$ normalizes each coordinate by its typical gradient size, so the step is roughly η per coordinate regardless of scale.

In practice Adam needs little tuning: $\eta = 10^{-3}$ is a good default, lowered to 10^{-4} once the loss plateaus. In deep hedging this is especially useful because the network weights and the CVaR threshold w live on different scales, and Adam adapts to both automatically.

In deep hedging.

- Each step simulates B fresh paths, which form the mini-batch, and discards them afterwards. There is no fixed dataset, hence no epochs; we count steps only. Because the network never sees the same path twice, overfitting is not a concern.
- For CVaR in the Rockafellar–Uryasev form, the loss is an expectation for fixed w, so the mini-batch gradient is unbiased. But only the worst $(1-\alpha)B$ paths contribute to it, so B must be large (e.g. $B \geq 10^4$ for $\alpha = 0.95$).

- For entropic risk, the loss $\frac{1}{\lambda}\log\mathbb{E}[e^{-\lambda X}]$ is not linear in the expectation, so the mini-batch gradient is slightly biased. The bias vanishes as $B \to \infty$ and is negligible at $B = 10^4$.
- The training loss keeps fluctuating at the end of training. This is the constant-step-size effect above, not a failure to converge. Judge progress on a fixed test set, and reduce η for the final phase.

4.3 Backpropagation

Gradients are computed by the chain rule, applied automatically (*autograd*). The forward pass records every operation; `loss.backward()` traverses the record in reverse and multiplies local derivatives.

Example 4.8 (Backprop by hand)**.** $f(x) = w_2\,\mathrm{ReLU}(w_1 x)$, loss $L = (f-y)^2$, with $x = 1$, $w_1 = 2$, $w_2 = 3$, $y = 5$.

Forward: $u = w_1 x = 2$, $h = \mathrm{ReLU}(u) = 2$, $f = w_2 h = 6$, $L = 1$.

Backward:

$$\frac{\partial L}{\partial f} = 2(f-y) = 2, \qquad \frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial f}\,h = 4, \qquad \frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial f}\,w_2\,\mathbf{1}\{u>0\}\,x = 6.$$

A gradient step with $\eta = 0.01$ gives $w_1 = 1.94$, $w_2 = 2.96$, so $f = 5.74$ and $L \approx 0.55$: the loss decreased.

The consequence for us: **any quantity computed from θ by differentiable tensor operations can be minimized**, including a P&L built by a 30-step loop. We never write a derivative by hand.

4.4 Practical rules

- **Scale inputs to $O(1)$.** Feed $\log(S/K)$, not $S \approx 100$. Badly scaled inputs saturate or stall training.
- **Learning rate.** Loss oscillates or explodes: too large. Loss barely moves: too small.
- **Overfitting** (train loss down, validation loss up) is almost impossible here, because every step uses fresh simulated paths. Still evaluate on a fixed test set, which makes numbers comparable across runs.
- **No dropout, no half precision.** Dropout makes the hedge random; float16 adds noise in exactly the tail that CVaR cares about.

5 Deep Hedging

5.1 From strategies to networks

Step 1: strategies as feedback rules. An admissible δ_k is a function of the information at t_k. In a Markov model, the relevant information is a low-dimensional state I_k, and we look for $\delta_k = f(I_k)$.

Step 2: choice of state.

Setting	State I_k	Reason
GBM, no costs	$\big(\log \frac{S_k}{K},\ T - t_k\big)$	BS delta depends only on these.
GBM, costs	add δ_{k-1}	The optimal trade depends on the current position.
Heston, two instruments	add V_k/b and $\delta_{k-1} \in \mathbb{R}^2$	Variance is a state variable.

Including δ_{k-1} is not optional with costs. With $c > 0$ the state of the hedging problem is (market state, current holdings), because the cost of moving to a target depends on where you are now.

Step 3: one network for all dates.

$$\delta_k = F_\theta(I_k), \qquad k = 0, \dots, n-1,$$

with the same θ at every step; the date enters through $T - t_k$. (The original paper uses a separate network per date. Sharing is simpler, has n times fewer parameters, and works well.)

Step 4: the training problem. Replace the infimum over all admissible strategies by an infimum over θ, and the expectation by a Monte Carlo average over B simulated paths:

$$\min_{\theta\,(,w)} \ \hat{\rho}\Big(\mathrm{PL}_T^{(1)}(\theta), \dots, \mathrm{PL}_T^{(B)}(\theta)\Big). \tag{3}$$

For CVaR, $\hat{\rho} = w + \frac{1}{(1-\alpha)B} \sum_i (-\mathrm{PL}_T^{(i)} - w)^+$.

Theorem 5.1 (Approximation; Buehler et al. 2019, informal)**.** *If the networks are allowed to grow, the optimal value of the network-restricted problem converges to the optimal value $\pi(-Z)$ over all admissible strategies.*

So restricting to networks loses nothing in the limit. The remaining errors are finite network size, finite sample size, and imperfect optimization, and we check them empirically against benchmarks.

5.2 The rollout

The network is called *sequentially* along each path, and its output at step k is an input at step $k+1$. In recursive form, with PL accumulated as we go:

$$\begin{aligned}
&\mathrm{PL} \leftarrow p_0 - Z, \qquad \delta_{-1} \leftarrow 0,\\
&\text{for } k = 0, \dots, n-1: \quad \delta_k = F_\theta(I_k), \quad \mathrm{PL} \leftarrow \mathrm{PL} + \delta_k \cdot (S_{k+1} - S_k) - c\, S_k \cdot |\delta_k - \delta_{k-1}|,\\
&\mathrm{PL} \leftarrow \mathrm{PL} - c\, S_n \cdot |\delta_{n-1}| \qquad \text{(liquidation)}.
\end{aligned}$$

Example 5.2 (Two-step rollout by hand)**.** $K = 100$, $c = 0.1\%$, path $S_0 = 100 \to S_1 = 102 \to S_2 = 99$. Suppose the network outputs $\delta_0 = 0.5$, $\delta_1 = 0.6$.

k	S_k	δ_k	trade	cost $c\,S_k\lvert\text{trade}\rvert$	gain $\delta_k(S_{k+1} - S_k)$
0	100	0.5	$+0.5$	0.0500	$0.5 \cdot (+2) = +1.0$
1	102	0.6	$+0.1$	0.0102	$0.6 \cdot (-3) = -1.8$
2	99	0 (liquidate)	-0.6	0.0594	–
				0.1196	-0.8

$Z = (99 - 100)^+ = 0$, so $\mathrm{PL}_T = p_0 - 0.8 - 0.1196 = p_0 - 0.9196$.

Use this exact example as a unit test for the rollout (Section 9).

5.3 How the gradient flows

Differentiating (1) with respect to a single position δ_k (scalar case, holding the other positions fixed):

$$\frac{\partial \mathrm{PL}_T}{\partial \delta_k} = \underbrace{(S_{k+1} - S_k)}_{\text{gain}} - \underbrace{c\, S_k \operatorname{sgn}(\delta_k - \delta_{k-1})}_{\text{cost of trading into } \delta_k} + \underbrace{c\, S_{k+1} \operatorname{sgn}(\delta_{k+1} - \delta_k)}_{\text{cost of trading out of } \delta_k}.$$

Two features of this formula explain the behavior of the trained hedger:

- The cost terms are $\pm cS$, independent of the trade size. Small adjustments pay a cost of first order, while the risk reduction they buy is second order. This is where the no-trade band (Section 7.2) comes from.
- There is also an *indirect* path: δ_k is an input to F_θ at step $k+1$, so it changes δ_{k+1}, which changes later gains and costs. Autograd follows this chain automatically, exactly as in a recurrent network.

The chain from PL_T back to θ passes through all n network calls. It breaks if anything inside the rollout calls `.item()`, `.numpy()`, `.detach()`, or runs under `torch.no_grad()`. The kink of $|\cdot|$ at 0 is not a problem: autograd uses the subgradient 0 there.

5.4 The algorithm

Algorithm 1 Deep hedging (training)

Require: simulator, payoff $Z(\cdot)$, cost rate c, risk measure $\hat{\rho}$, batch size B, steps N

Initialize network F_θ; for CVaR also $w \leftarrow 0$
Simulate a fixed test set of paths (fixed seed)
for $j = 1, \dots, N$ **do**
  Simulate B fresh paths $S^{(i)}_{0:n}$ (and $V^{(i)}_{0:n}$)
  $\mathrm{PL} \leftarrow -Z(S_{0:n})$; $\delta_{\mathrm{prev}} \leftarrow 0$
  for $k = 0, \dots, n-1$ **do**
    $I_k \leftarrow \texttt{features}(S_k, V_k, t_k, \delta_{\mathrm{prev}})$ ▷ shape (B, d)
    $\delta_k \leftarrow F_\theta(I_k)$ ▷ shape (B, m)
    $\mathrm{PL} \leftarrow \mathrm{PL} + \delta_k \cdot (S_{k+1} - S_k) - c\, S_k \cdot |\delta_k - \delta_{\mathrm{prev}}|$
    $\delta_{\mathrm{prev}} \leftarrow \delta_k$
  end for
  $\mathrm{PL} \leftarrow \mathrm{PL} - c\, S_n \cdot |\delta_{\mathrm{prev}}|$
  loss $\leftarrow \hat{\rho}(\mathrm{PL})$
  Backpropagate; Adam step on (θ, w)
  if j is a multiple of the evaluation interval **then**
    Evaluate on the test set (no gradient); log metrics; compare with benchmarks
  end if
end for

Cost per step. $B \cdot n$ forward passes of a network with a few thousand parameters. With $B = 10^4$ and $n = 30$, one step takes milliseconds on a laptop CPU, and a full run takes minutes.

6 Market Simulators

6.1 Geometric Brownian motion

With $r = 0$ and $\mathbb{P} = \mathbb{Q}$ (drift 0), sample exactly:

$$S_{k+1} = S_k \exp\Big(-\tfrac{1}{2}\sigma^2 \Delta t + \sigma\sqrt{\Delta t}\,\varepsilon_k\Big), \qquad \varepsilon_k \overset{\text{iid}}{\sim} N(0,1).$$

There is no discretization error, because the log-price increments are exactly Gaussian. This is the sanity-check model: the answer (BS delta) is known.

Remark 6.1 (Why zero drift). If $\mu \neq 0$, the quadratic-optimal discrete hedge is not the BS delta (it includes a correction from the mean of ΔS), and the network will learn to exploit the drift. With $\mu = 0$, S is a martingale, and the variance-optimal hedge converges to the BS delta as $\Delta t \to 0$. This gives a clean test.

6.2 Heston

Dynamics (zero rate, risk-neutral; this is the parametrization in the repo):

$$dS_t = S_t\sqrt{V_t}\,dB_t, \qquad dV_t = \alpha(b - V_t)\,dt + \sigma\sqrt{V_t}\,dW_t, \qquad d\langle B, W\rangle_t = \rho\,dt.$$

Parameters from the paper: $\alpha = 1$, $b = 0.04$, $\rho = -0.7$, $\sigma = 2$, $V_0 = 0.04$, $S_0 = 100$, $n = 30$ daily steps.

Discretization: full-truncation Euler with substep h, writing $V^+ = \max(V, 0)$:

$$S_{t+h} = S_t \exp\Big(-\tfrac{1}{2}V_t^+ h + \sqrt{V_t^+ h}\,Z^S\Big),$$
$$V_{t+h} = V_t + \alpha(b - V_t^+)h + \sigma\sqrt{V_t^+ h}\,Z^V, \qquad Z^S = \rho Z^V + \sqrt{1-\rho^2}\,Z^\perp.$$

Feller condition: $2\alpha b \geq \sigma^2$ guarantees $V > 0$. Here $2\alpha b = 0.08 \ll \sigma^2 = 4$, so V hits 0 often and discretization error is not negligible. Keep the substeps (the repo uses 10 per day), and mention it in the report.

6.3 The variance swap as a second instrument

To hedge volatility risk we need a traded instrument that depends on V. Take the variance swap whose value is the conditional expected realized variance:

$$S_t^{(2)} = \mathbb{E}\Big[\int_0^T V_s\,ds \,\Big|\, \mathcal{F}_t\Big] = \int_0^t V_s\,ds + L(t, V_t).$$

By construction $S^{(2)}$ is a $\mathbb{Q}$-martingale (a conditional expectation of one fixed random variable), so it is a legitimate zero-rate price process.

Proposition 6.2. $L(t, v) = \dfrac{v - b}{\alpha}\big(1 - e^{-\alpha(T-t)}\big) + b\,(T - t)$.

Proof. Taking expectations in the variance SDE (the stochastic integral has mean zero) gives the ODE $\frac{d}{ds}\mathbb{E}[V_s \mid V_t] = \alpha\big(b - \mathbb{E}[V_s \mid V_t]\big)$, so $\mathbb{E}[V_s \mid V_t = v] = b + (v - b)e^{-\alpha(s-t)}$. By Fubini,

$$L(t, v) = \int_t^T \Big(b + (v - b)e^{-\alpha(s-t)}\Big)ds = b(T - t) + \frac{v - b}{\alpha}\big(1 - e^{-\alpha(T-t)}\big). \qquad \square$$

Scaling. $S^{(1)} \approx 100$ while $S^{(2)} \approx bT = 0.04 \cdot 30/365 \approx 0.0033$. Positions in the second instrument will therefore be numerically large: roughly $\partial_v C / \partial_v L$, where $\partial_v L = (1 - e^{-\alpha\tau})/\alpha \approx \tau$. Either rescale the instrument (quote it per unit of bT) or let the network output a scaled position. Otherwise the network barely learns the second instrument.

7 Benchmarks and What to Expect

7.1 GBM without costs: recover the BS delta

With σ known and $r = 0$, $\tau = T - t$:

$$d_1 = \frac{\log(S/K) + \frac{1}{2}\sigma^2\tau}{\sigma\sqrt{\tau}}, \qquad \Delta^{BS} = N(d_1), \qquad C^{BS} = S\,N(d_1) - K\,N(d_1 - \sigma\sqrt{\tau}).$$

Example 7.1. $S = 105$, $K = 100$, $\tau = 10/365$, $\sigma = 0.2$: $d_1 = \frac{0.04879 + 0.00055}{0.2 \cdot 0.16552} = 1.490$, so $\Delta^{BS} = N(1.490) \approx 0.932$. The trained network at input $(\log 1.05,\ 10/365)$ should output approximately 0.93.

Pass criterion (M0). Plot δ_k against S_k at a few fixed t_k, with $N(d_1)$ on top. The curves should overlap, and the P&L standard deviation of the network hedge should be close to that of the BS delta hedge. If not, there is a bug. No experiment beyond this point is meaningful until M0 passes.

7.2 GBM with costs: the no-trade band

From the gradient formula in Section 5: a small trade costs $cS|\Delta\delta|$ (first order), while the hedging-error reduction it buys is second order in the deviation. So it is optimal *not* to trade while the position is close enough to the frictionless target, and when it drifts outside, to trade only back to the *edge* of the band.

Whalley–Wilmott asymptotics. For exponential utility with risk aversion γ and small c, the optimal band is approximately

$$\delta \in \left[\Delta^{BS} - H,\ \Delta^{BS} + H\right], \qquad H = \left(\frac{3\,c\,S\,\Gamma^2}{2\gamma}\right)^{1/3}, \qquad \Gamma = \frac{N'(d_1)}{S\sigma\sqrt{\tau}}.$$

Qualitative predictions to check against the network:

- width scales like $c^{1/3}$, not c;
- the band is widest where Γ is large: near the money and close to expiry;
- plotting δ_k against δ_{k-1} (other inputs fixed) shows a flat-then-diagonal-then-flat shape: inside the band the output equals the input (no trade).

These are asymptotic results for continuous trading. The network trades discretely and uses CVaR, so expect agreement in shape, not exact numbers.

7.3 Heston

- **Spot only.** Benchmark: BS delta evaluated with the current volatility $\sqrt{V_k}$. Vol risk is unhedged, so the P&L has fat tails and the network should beat the benchmark mainly in the tail.
- **Spot plus variance swap.** The market is (approximately) complete in continuous time. The model hedge is

$$\delta^{(1)} = \partial_S C^H, \qquad \delta^{(2)} = \frac{\partial_v C^H}{\partial_v L} = \frac{\alpha\, \partial_v C^H}{1 - e^{-\alpha(T-t)}},$$

  with $C^H(t, S, v)$ the Heston call price (computable with Carr–Madan). Expect a large reduction in CVaR compared with spot only.

7.4 Indifference price

With $c = 0$ in GBM, p_0 should be close to C^{BS}, up to discrete-hedging error. With $c > 0$, $p_0 > C^{BS}$: the excess is the compensation for costs and residual risk, and it increases with c and with risk aversion.

8 Experiments and Evaluation

ID	Model	Objective	Question
M0	GBM, $c = 0$	quadratic	Does the network recover $N(d_1)$?
M1	GBM, $c = 0$	$\text{CVaR}_{0.95}$	Same hedge? Better tail?
M2	GBM, $c \in \{0.05, 0.1, 0.5\}\%$	$\text{CVaR}_{0.95}$	No-trade band; beats BS delta?
M3	Heston, spot only	$\text{CVaR}_{0.95}$	Gain over BS delta with $\sqrt{V_k}$
M4	Heston, spot + var swap	$\text{CVaR}_{0.95}$	Tail reduction vs. M3
P	any	$\text{CVaR}_{0.95}$	Indifference price $\pi(-Z) - \pi(0)$

Metrics (always on the fixed test set, for both network and benchmark): mean, standard deviation, $\text{CVaR}_{0.95}$ of PL_T; average turnover $\mathbb{E}\sum_k |\delta_k - \delta_{k-1}|$; total cost $\mathbb{E}[C_T]$; indifference price.

Plots. P&L histograms (network vs. benchmark, overlaid); δ vs. S at fixed t_k; δ_k vs. δ_{k-1} for the band; training curve (train loss and test loss).

Reliability. Run 3–5 seeds and report the spread. Use batch size $\geq 10^4$ for $\text{CVaR}_{0.95}$, so that each batch has at least 500 tail paths.

9 Implementation Guide

9.1 Repository map

File	Content	Notes section
`market/simulators.py`	GBM, Heston + variance swap	6
`market/pricing.py`	BS price/delta/gamma (Heston via Carr–Madan, optional)	7
`liabilities.py`	payoffs $Z(S_{0:n})$	1
`features.py`	state I_k, normalized	5
`model.py`	MLP policy F_θ	4
`frictions.py`	cost $c\,S_k \cdot \lvert\delta_k - \delta_{k-1}\rvert$	1
`objectives.py`	quadratic, entropic, CVaR with learnable w	3
`train.py`	rollout, training loop, evaluation	5
`benchmarks.py`	BS delta, BS with $\sqrt{V_k}$, Whalley–Wilmott band	7

9.2 Interfaces

B = number of paths, n = time steps, m = instruments, d = features.

Function	Returns
`simulate(..., generator)`	dict: `S` (B,n+1,m), `V` (B,n+1), `T`, `dt`
`payoff(S)`	`(B,)`
`features(S, V, k, delta_prev, cfg)`	`(B,d)`
`policy(I)`	`(B,m)`
`cost(delta, delta_prev, S_k)`	`(B,)`
`rho(PL)`	scalar
`hedge_pnl(policy, paths, payoff, cost)`	`(B,)`

The benchmarks implement the same `policy` signature as the network, so one `hedge_pnl` evaluates both. This guarantees the comparison uses identical P&L accounting.

9.3 Tests

1. **Simulator (GBM).** Monte Carlo call price within 3 standard errors of C^{BS}.
2. **Simulator (Heston).** $\mathbb{E}[S_n^{(2)}] \approx S_0^{(2)}$ (martingale), and $\mathbb{E}[S_n^{(1)}] \approx S_0$.
3. **Rollout.** The two-step example of Section 5: a constant policy returning 0.5 then 0.6 on the path $100 \to 102 \to 99$ gives $\mathrm{PL}_T = p_0 - 0.9196$.
4. **Benchmark.** BS delta hedge, GBM, $c = 0$: P&L standard deviation decreases roughly like $1/\sqrt{n}$ as n grows.
5. **Objectives.** The CVaR example of Section 3: losses $\{-1, 0, 2, 3, 6\}$, $\alpha = 0.6$, value 4.5.

9.4 Debugging checklist

- **Shapes.** `(B,1)` minus `(B,)` broadcasts silently to `(B,B)`. Squeeze or keep `(B,m)` consistently and assert shapes.
- **Gradient chain.** No `.item()`, `.numpy()`, `.detach()`, or `no_grad` inside the rollout. Call `optimizer.zero_grad()` every step.
- **Loss flat.** Check input scaling, then lower the learning rate. Plot δ vs. S: a constant output means the network is not using its inputs.
- **Off-by-one.** S has $n+1$ dates; δ has n. The liquidation cost uses S_n.
- **Large c.** The network may learn "never trade". Compare with the benchmark CVaR before calling it a bug; it can be optimal.
- **Tail noise.** CVaR loss jumps around: increase B.

10 Limitations and Extensions

- **In-model results.** Training and testing on the same simulator flatters the method. The real question is performance when the simulator is wrong: train on Heston, test on rough volatility or on bootstrapped historical paths.
- **European payoffs, $r = 0$.** American options need $r > 0$ (otherwise early exercise of a put is never optimal) and a model of the holder's exercise rule; the liability is then settled at a random time.
- **No uniqueness.** Different seeds can give different strategies with similar risk, especially with costs, where the objective is flat near the optimum.
- **Choice of ρ.** Results depend on α and λ. Report them, and ideally show sensitivity.

References

[1] H. Buehler, L. Gonon, J. Teichmann, B. Wood. Deep hedging. *Quantitative Finance* 19(8), 2019.

[2] R. T. Rockafellar, S. Uryasev. Optimization of conditional value-at-risk. *Journal of Risk* 2(3), 2000.

[3] A. E. Whalley, P. Wilmott. An asymptotic analysis of an optimal hedging model for option pricing with transaction costs. *Mathematical Finance* 7(3), 1997.

[4] K. Hornik. Approximation capabilities of multilayer feedforward networks. *Neural Networks* 4(2), 1991.

[5] D. P. Kingma, J. Ba. Adam: a method for stochastic optimization. ICLR, 2015.

[6] H. Föllmer, A. Schied. *Stochastic Finance: An Introduction in Discrete Time*. De Gruyter, 4th ed., 2016.